

1. Find Missing Number in an Array

- **Logic:** The sum of the first n natural numbers is $n * (n + 1) / 2$. The missing number will be the difference between the expected sum and the actual sum of the array.

```
public class MissingNumber {
    public static void main(String[] args) {
        int[] arr = {1, 2, 4, 5, 6};
        int n = 6; // Assuming the array should contain numbers from
1 to n
        int expectedSum = n * (n + 1) / 2;
        int actualSum = 0;
        for (int num : arr) {
            actualSum += num;
        }
        int missingNumber = expectedSum - actualSum;
        System.out.println("Missing number: " + missingNumber);
    }
}
```

- **Flowchart:** Input array → Calculate sum of numbers from 1 to n → Calculate sum of array → Subtract actual sum from expected → Output missing number.
- **Time Complexity:** $O(n)$, where n is the size of the array.
- **Space Complexity:** $O(1)$

2. Search an Element in a Sorted and Rotated Array

- **Logic:** Use binary search, but modify it to handle the rotation by checking which part of the array is sorted and narrowing down the search space accordingly.

```
public class SearchInRotatedArray {
    public static int search(int[] arr, int target) {
        int left = 0, right = arr.length - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (arr[mid] == target) return mid;
            if (arr[left] <= arr[mid]) {
                if (arr[left] <= target && target < arr[mid]) right
= mid - 1;
            } else {
                left = mid + 1;
            }
        }
    }
}
```

```

        if (arr[mid] < target && target <= arr[right]) left
= mid + 1;
        else right = mid - 1;
    }
}
return -1;
}

public static void main(String[] args) {
    int[] arr = {6, 7, 9, 15, 19, 2, 3};
    int target = 15;
    System.out.println("Element found at index: " + search(arr,
target));
}
}

```

- **Flowchart:** Input array and target → Perform binary search considering rotated array → Return index if element found, else -1.
- **Time Complexity:** $O(\log n)$
- **Space Complexity:** $O(1)$

3. Find Second Largest Number in Array

- **Logic:** Traverse the array once, keeping track of the largest and second largest numbers.

```

public class SecondLargest {
    public static void main(String[] args) {
        int[] arr = {2, 3, 1, 5, 4};
        int largest = Integer.MIN_VALUE, secondLargest =
Integer.MIN_VALUE;
        for (int num : arr) {
            if (num > largest) {
                secondLargest = largest;
                largest = num;
            } else if (num > secondLargest && num != largest) {
                secondLargest = num;
            }
        }
        System.out.println("Second largest number: " +
secondLargest);
    }
}

```

```
}
```

- **Flowchart:** Input array → Loop through array → Update largest and second largest → Output second largest.
- **Time Complexity:** $O(n)$
- **Space Complexity:** $O(1)$

4. Find Number Occurring Odd Number of Times in Array

- **Logic:** XOR all elements. The result will be the element that occurs an odd number of times because XOR cancels out pairs.

```
public class OddOccurrences {
    public static void main(String[] args) {
        int[] arr = {4, 3, 4, 5, 3, 5, 3};
        int result = 0;
        for (int num : arr) {
            result ^= num;
        }
        System.out.println("Number occurring odd times: " + result);
    }
}
```

- **Flowchart:** Input array → XOR all elements → Output result.
- **Time Complexity:** $O(n)$
- **Space Complexity:** $O(1)$

5. Minimum Numbers of Platforms Required for Railway Station

- **Logic:** Sort the arrival and departure times. Use two pointers to track the number of platforms needed at any time.

```
import java.util.*;

public class RailwayPlatform {
    public static int findPlatform(int[] arrival, int[] departure) {
        Arrays.sort(arrival);
        Arrays.sort(departure);
        int platforms = 1, result = 1;
        int i = 1, j = 0;
        while (i < arrival.length && j < departure.length) {
            if (arrival[i] <= departure[j]) {
                platforms++;
            }
        }
    }
}
```

```

        i++;
    } else {
        platforms--;
        j++;
    }
    result = Math.max(result, platforms);
}
return result;
}

public static void main(String[] args) {
    int[] arrival = {1000, 1030, 1045};
    int[] departure = {1030, 1100, 1115};
    System.out.println("Minimum platforms required: " +
findPlatform(arrival, departure));
}
}

```

- **Flowchart:** Input arrival and departure times → Sort both arrays → Use two pointers to calculate minimum platforms → Output result.
- **Time Complexity:** $O(n \log n)$
- **Space Complexity:** $O(1)$

6. Find Pair Whose Sum is Closest to Zero

- **Logic:** Sort the array, then use two pointers to find the pair with the smallest sum.

```

import java.util.*;

public class ClosestToZero {
    public static void closestPair(int[] arr) {
        Arrays.sort(arr);
        int left = 0, right = arr.length - 1;
        int closestSum = Integer.MAX_VALUE;
        int leftNum = 0, rightNum = 0;

        while (left < right) {
            int sum = arr[left] + arr[right];
            if (Math.abs(sum) < Math.abs(closestSum)) {
                closestSum = sum;
                leftNum = arr[left];
                rightNum = arr[right];
            }
            if (sum < 0) left++;
            else right--;
        }
    }
}

```

```

        }
        if (sum < 0) left++;
        else right--;
    }

    System.out.println("Pair with sum closest to zero: " +
leftNum + ", " + rightNum);
}

public static void main(String[] args) {
    int[] arr = {-1, 2, 3, -4, 5};
    closestPair(arr);
}
}

```

- **Flowchart:** Input array → Sort array → Use two pointers to find closest sum → Output pair.
- **Time Complexity:** $O(n \log n)$ due to sorting.
- **Space Complexity:** $O(1)$

7. Find Pair Whose Sum is Closest to X in Array

- **Logic:** Similar to the above, but instead of zero, use X as the target sum.

```

public class ClosestToX {
    public static void closestPair(int[] arr, int X) {
        Arrays.sort(arr);
        int left = 0, right = arr.length - 1;
        int closestSum = Integer.MAX_VALUE;
        int leftNum = 0, rightNum = 0;

        while (left < right) {
            int sum = arr[left] + arr[right];
            if (Math.abs(sum - X) < Math.abs(closestSum - X)) {
                closestSum = sum;
                leftNum = arr[left];
                rightNum = arr[right];
            }
            if (sum < X) left++;
            else right--;
        }
    }
}

```

```

        System.out.println("Pair with sum closest to " + X + ": " +
leftNum + ", " + rightNum);
    }

    public static void main(String[] args) {
        int[] arr = {-1, 2, 3, -4, 5};
        int X = 3;
        closestPair(arr, X);
    }
}

```

- **Flowchart:** Input array and X → Sort array → Use two pointers to find closest sum to X → Output pair.
- **Time Complexity:** $O(n \log n)$
- **Space Complexity:** $O(1)$

8. Find All Pairs of Elements Whose Sum is Equal to Given Number

- **Logic:** Use a hash set to track the visited numbers and check if the complement exists.

```

import java.util.*;

public class AllPairsWithSum {
    public static void findPairs(int[] arr, int sum) {
        Set<Integer> set = new HashSet<>();
        for (int num : arr) {
            int complement = sum - num;
            if (set.contains(complement)) {
                System.out.println("Pair: (" + num + ", " +
complement + ")");
            }
            set.add(num);
        }
    }

    public static void main(String[] args) {
        int[] arr = {1, 4, 5, 6, 7, 8};
        int sum = 10;
        findPairs(arr, sum);
    }
}

```

- **Flowchart:** Input array and sum → Use hash set to find pairs → Output pairs.
- **Time Complexity:** $O(n)$
- **Space Complexity:** $O(n)$

9. Search Element in Row-wise and Column-wise Sorted Matrix

- **Logic:** Start from the top-right corner and move left or down based on comparisons with the target.

```
public class SearchInMatrix {
    public static boolean search(int[][] matrix, int target) {
        int row = 0, col = matrix[0].length - 1;
        while (row < matrix.length && col >= 0) {
            if (matrix[row][col] == target) return true;
            else if (matrix[row][col] > target) col--;
            else row++;
        }
        return false;
    }

    public static void main(String[] args) {
        int[][] matrix = {
            {1, 2, 3, 4},
            {5, 6, 7, 8},
            {9, 10, 11, 12}
        };
        int target = 10;
        System.out.println("Element found: " + search(matrix,
target));
    }
}
```

- **Flowchart:** Input matrix and target → Start at top-right → Move left or down → Return true if found.
- **Time Complexity:** $O(m + n)$, where m is number of rows and n is number of columns.
- **Space Complexity:** $O(1)$

10. Program to Implement Stack Using Array

- **Logic:** Use an array to implement the stack, with push and pop operations.

```
public class StackUsingArray {
```

```

static final int MAX = 1000;
int top = -1;
int[] stack = new int[MAX];

boolean push(int x) {
    if (top >= MAX - 1) {
        System.out.println("Stack Overflow");
        return false;
    } else {
        stack[++top] = x;
        return true;
    }
}

int pop() {
    if (top < 0) {
        System.out.println("Stack Underflow");
        return -1;
    } else {
        return stack[top--];
    }
}

public static void main(String[] args) {
    StackUsingArray stack = new StackUsingArray();
    stack.push(10);
    stack.push(20);
    System.out.println("Popped element: " + stack.pop());
}
}

```

- **Flowchart:** Input data → Push/pop elements from array → Return element or error message.
- **Time Complexity:** $O(1)$ for both push and pop operations.
- **Space Complexity:** $O(n)$

11. Program to Implement Stack Using Linked List

- **Logic:** A linked list is used where each node contains data and a reference to the next node. The stack operations (push and pop) are performed by manipulating the head pointer.

```

public class StackUsingLinkedList {
    Node top;

    static class Node {
        int data;
        Node next;

        Node(int data) {
            this.data = data;
            next = null;
        }
    }

    public boolean push(int x) {
        Node newNode = new Node(x);
        newNode.next = top;
        top = newNode;
        return true;
    }

    public int pop() {
        if (top == null) {
            System.out.println("Stack Underflow");
            return -1;
        } else {
            int popped = top.data;
            top = top.next;
            return popped;
        }
    }

    public static void main(String[] args) {
        StackUsingLinkedList stack = new StackUsingLinkedList();
        stack.push(10);
        stack.push(20);
        System.out.println("Popped element: " + stack.pop());
    }
}

```

- **Flowchart:** Input data → Create new node → Set next pointer → Update top → Pop element by updating top pointer.

- **Time Complexity:** $O(1)$ for both push and pop operations.
 - **Space Complexity:** $O(n)$ (due to linked list nodes)
-

12. Program to Implement Stack Using Two Queues

- **Logic:** Implement stack behavior using two queues. When pushing, elements are enqueued to the first queue. For popping, the first queue is dequeued into the second queue until one element remains.

```
import java.util.*;

public class StackUsingTwoQueues {
    Queue<Integer> q1 = new LinkedList<>();
    Queue<Integer> q2 = new LinkedList<>();

    public void push(int x) {
        q1.add(x);
    }

    public int pop() {
        if (q1.isEmpty()) {
            System.out.println("Stack Underflow");
            return -1;
        }
        while (q1.size() > 1) {
            q2.add(q1.remove());
        }
        int popped = q1.remove();
        Queue<Integer> temp = q1;
        q1 = q2;
        q2 = temp;
        return popped;
    }

    public static void main(String[] args) {
        StackUsingTwoQueues stack = new StackUsingTwoQueues();
        stack.push(10);
        stack.push(20);
        System.out.println("Popped element: " + stack.pop());
    }
}
```

- **Flowchart:** Input data → Add to queue q1 → Pop by transferring elements to q2 → Return the popped element.
 - **Time Complexity:** $O(n)$ for pop, $O(1)$ for push.
 - **Space Complexity:** $O(n)$
-

13. Sort a Stack Using Another Stack

- **Logic:** Use a temporary stack to sort the elements of the original stack. Pop elements from the original stack and push them to the temporary stack in sorted order.

```
import java.util.*;

public class SortStack {
    public static void sortStack(Stack<Integer> stack) {
        Stack<Integer> tempStack = new Stack<>();
        while (!stack.isEmpty()) {
            int temp = stack.pop();
            while (!tempStack.isEmpty() && tempStack.peek() > temp)
            {
                stack.push(tempStack.pop());
            }
            tempStack.push(temp);
        }
        while (!tempStack.isEmpty()) {
            stack.push(tempStack.pop());
        }
    }

    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<>();
        stack.push(3);
        stack.push(1);
        stack.push(4);
        stack.push(2);
        sortStack(stack);
        System.out.println("Sorted stack: " + stack);
    }
}
```

- **Flowchart:** Input stack → Pop element from stack → Sort and push into temp stack → Transfer back to original stack.
 - **Time Complexity:** $O(n^2)$
 - **Space Complexity:** $O(n)$
-

14. Implement Queue Using Arrays

- **Logic:** A queue can be implemented using a fixed-size array, where **front** and **rear** pointers are used to track the queue's elements.

```
public class QueueUsingArray {
    static final int MAX = 1000;
    int front = -1, rear = -1;
    int[] queue = new int[MAX];

    boolean enqueue(int x) {
        if (rear >= MAX - 1) {
            System.out.println("Queue Overflow");
            return false;
        } else {
            if (front == -1) front = 0;
            queue[++rear] = x;
            return true;
        }
    }

    int dequeue() {
        if (front == -1 || front > rear) {
            System.out.println("Queue Underflow");
            return -1;
        } else {
            return queue[front++];
        }
    }

    public static void main(String[] args) {
        QueueUsingArray queue = new QueueUsingArray();
        queue.enqueue(10);
        queue.enqueue(20);
        System.out.println("Dequeued element: " + queue.dequeue());
    }
}
```

```
}
```

- **Flowchart:** Input data → Enqueue to rear → Dequeue from front → Return dequeued element.
 - **Time Complexity:** $O(1)$ for both enqueue and dequeue operations.
 - **Space Complexity:** $O(n)$
-

15. Implement Queue Using Linked List

- **Logic:** Use a linked list where each node stores data and a reference to the next node. The **enqueue** operation adds to the tail, and the **dequeue** operation removes from the head.

```
public class QueueUsingLinkedList {
    Node front, rear;

    static class Node {
        int data;
        Node next;
        Node(int data) {
            this.data = data;
            next = null;
        }
    }

    public boolean enqueue(int x) {
        Node newNode = new Node(x);
        if (rear == null) {
            front = rear = newNode;
            return true;
        }
        rear.next = newNode;
        rear = newNode;
        return true;
    }

    public int dequeue() {
        if (front == null) {
            System.out.println("Queue Underflow");
            return -1;
        }
    }
}
```

```

        int dequeued = front.data;
        front = front.next;
        if (front == null) rear = null;
        return dequeued;
    }

    public static void main(String[] args) {
        QueueUsingLinkedList queue = new QueueUsingLinkedList();
        queue.enqueue(10);
        queue.enqueue(20);
        System.out.println("Dequeued element: " + queue.dequeue());
    }
}

```

- **Flowchart:** Input data → Create node and enqueue → Dequeue from front → Return dequeued element.
- **Time Complexity:** $O(1)$ for both enqueue and dequeue operations.
- **Space Complexity:** $O(n)$ (due to linked list nodes)

Summary of Time and Space Complexity for Each:

1. **Stack Using Array:**
 - Time Complexity: $O(1)$
 - Space Complexity: $O(n)$
2. **Stack Using Linked List:**
 - Time Complexity: $O(1)$
 - Space Complexity: $O(n)$
3. **Stack Using Two Queues:**
 - Time Complexity: $O(n)$ for pop, $O(1)$ for push
 - Space Complexity: $O(n)$
4. **Sort Stack Using Another Stack:**
 - Time Complexity: $O(n^2)$
 - Space Complexity: $O(n)$
5. **Queue Using Arrays:**
 - Time Complexity: $O(1)$
 - Space Complexity: $O(n)$
6. **Queue Using Linked List:**
 - Time Complexity: $O(1)$
 - Space Complexity: $O(n)$

=====